Homepage: http://www.vim.org/index.php
Plugins: http://www.vim.org/scripts/script_search_results.php?order_by=downloads

Resources
Video: https://www.youtube.com/watch?v=SkdrYWhh-8s
Useful commands: http://zzapper.co.uk/vimtips.html

Sample dot files
Sensible vim: https://github.com/tpope/vim-sensible
https://github.com/r00k/dotfiles
https://github.com/chrishunt/dot-files
https://github.com/sheerun/dotfiles

Variables

```
" Print variable
:echo $MYVIMRC

" Common variables
echo $MYVIMRC                Current .vimrc path
echo $VIMRUNTIME             Runtime files installed with vim.
echo b:current_syntax        Currently set syntax file
```

Wildcard
:help wildcard

?
* match anything, including nothing
** match anything, including nothing. Recurse into subdirectory upto 100 deep
[abc] One character

Examples
???.js File names with 3 characters e.g. ab.js, cd.js
*.js Any js file. e.g. type.js, MyClass.js
**/*.js Any js file, including subdirs. e.g. file.js, a/file.js, a/b/c/file.js

[abc] a,b,c
[a-z]

Help

```
"" Help keys
:help help.txt      Help for keys
g0                  Show table of contents (0=big o)
<c-]>               Follow the link.
<c-t>               Back
<c-o>               Go back to old location.
<c-i>               Go to new location.


"" Basics
:help help.txt        Main help page
:h help.txt           Same
:vert help            Open in vertical tab



" Main help pages
```

```
:h help.txt            List of all main help pages
:h options             List of all options
:h index               List of all commands and maps
:h ex-commands         List of all EX commands
:h pattern.txt         Search and replace
:h help-summary        Summary of help syntax
:h script              Scripting help


"" Help by feature
"
" i.e. as specified in :help help-summary
"
" help for vim options
:h options.txt            Main options help
:h option-summary         List of all options
:h 'tabstop'              Specific option e.g. 'tabstop' option
:h 'number'               e.g. number option

" help for vim commands
:h ex-cmd-index           list of all commands
:h :echo                  Specify option e.g. :echo command
:h :au                    e.g. autocmd command

: help for vim functions
:h builtin-functions      List of all functions
:h abs()                  Specify function e.g. abs() function
:h getcwd()               e.g. getcwd() function

" help for mappings
:h index                 List of all maps
:h i                     Specify normal mode map e.g. i=insert
:h ctrl-d                e.g. ctrl-d=scroll down

" insert mode mappings
:h insert-index          List of all insert mode maps
:h i_<ESC>               Specify insert mode map e.g. <ESC>=exit insert mode
:h i_ctrl-x_ctrl-n

" visual mode mappings
:h visual-index          List of all visual mode maps
:h v_d
:h v_<ESC>

" Common map syntax
:h index          See list of all maps for details
:h i              key map e.g. i=insert
:h x              e.g. x=delete under cursor
:h [              e.g. [ motion
:h ]
:h $              symbol map e.g. $=jump to end of line
:h ^              e.g. ^=jump to start of line
:h g              list of all g* maps
:h gg             g map e.g. gg=beginning of line
:h CTRL-d         Control key
:h ctrl-d         same
:h ctrl-w_=       e.g. <c-w>=  make all windows equal size
:h <Tab>
:h <Space>
:h <ESC>

" help by feature type
:h 'tabstop'       Option help e.g. 'tabstop' option.
```

```
:h :echo           Command help e.g. :echo command
:h abs()           Function help e.g. abs() function
:h filetype        Plugin/feature help e.g. filetype feature

" Help for mappings
:h CTRL-R          Help for <C-R> in normal mode
:h ctrl-r          same
:h i_CTRL-R        Insert mode
:h c_CTRL-R        Command mode
:h v_CTRL-R        Visual mode
```

Runtime path and files

```
"" runtime path
:set runtimepath?               See runtime path
:set runtimepath+=~/myruntime   Append to path

"" :runtime[!] [where] file...
" Reload runtime files under runtime paths.
" By default, only the first found file is sourced. ! sources all found.
" Non existent files are ignored.
:runtime filetype.vim           Reload first encountered <runtimepath>/
filetype.vim
:runtime syntax/c.vim           <runtimepath>/syntax/c.vim
:runtime syntax/*.vim           All syntax files.
:runtime! filetype.vim          Reload all <runtimepath>/filetype.vim
```

==========

Motions

Motion keys

```
:help motion.txt
<c-g>         Show current cursor location and file information
g <c-g>       Show column, line, word number

h      left
j      down
k     up
l      right

0      start of the line
^      start of the line, skipping blanks
$     end of the current line, including the last character.
```

When line is wrapped, can move by screen line instead of logical line.

```
gj         Down one screen line
gk         Up
g0         Start of line
g^
g$


f s     First 's' (cursor on 's', search to right)
F s     Search to left
t s     To the 's' (cursor before 's')
T s     Search to left
;       Repeat
,       Opposite direction

w     Beginning of next word.
W    Beginning of next WORD (separated by white space)
```

```
e     End of current word
E     End of current WORD

b     Beginning of last word.
B     Beginning of last WORD.
ge    End of last word
gE    End of last WORD


gg      beginning of file
nG      n th line
n |     Nth column
G       end of file

[{            Jump to nearest {. See :help [{
]}            Nearest }
[(            Nearest (
])            Nearest )


<c-u>     half screen up
<c-d>     half screen down
<c-f>      Scroll up
<c-b>     Scroll down


:jumps       Show jump list
<c-o>        Jump to old
<c-i>        Jump to next
```

Text object
a = around, i = inside

```
:help objects

aw         Around word. Include space after word.
iw         In word. Do not include space.
aW         Around WORD
iW
as         Around sentence
is
ap         Around paragraph
ip
at         Around html tag. e.g. dit in <div>content</div> Deletes 'content'
it

a"         Include ""
i"         Inside ""
a'
i'
a`
i`
a(          Around ()
i(
a{          Around {}
i{
a[          Around []
i[
a<          Around <>
```

== Examples ==

- combine verb (e.g. change, delete) with noun (e.g. in word, around quotes)
  e.g. ci" change in double quotes quotes

```
viW     Visually select WORD
ci"     Change text in double quotes
di"     Delete text inside double quotes
```

==========

Basic usage

Insert and delete

```
i     Insert before selected character
a     Insert after selected character (append)
I     Same as ^i. Insert at beggining of line
A     Same as $a. Append at the end of the line

o     open a new line below the cursor.
O     open a new line above the cursor.

c           change (delete and stay in insert mode)
c{motion}
cl          character under cursor
s           same
ch          character to left.
cw          word

C           change to end of line
c$          same

S           change whole line
cc          same

r{char}        replace selected character with x (replace)
R{string}      replaces more than one character with new text

d{motion}      delete
x              delete character under cursor
dl             Same

D              delete to end of line
d$             Same

dd            delete whole line
2dd           delete 2 lines
```

Formatting

```
gU          Upper case
gu           Lower case
g~           Switch cases

# Format lines:
# By default, applies 'textwidth'
# If 'textwidth=0" (default), uses 79 width.
# If 'formatexpr' option is set, apply that
gq                Format the lines that cursor moves over
{visual}gq    Format visual selection
gqq              Format cu
```

== Examples ==

```
gqap       Format paragraph

ga           Print key of character below cursor.
```

Yank and paste

```
p     Put after cursor.
      Cursor moved to last character of new text
P     Put under cursor.

gp    Same as p, except cursor moved to after last character of new text.
gP

y     yanks the selected text in visual mode.
y{motion}
```

== Examples ==

```
:put 1          Put register 1 under cursor
:10put 1      Put register 1 at line 10
:Put 1
:10Put 1
```

Undo and redo

```
u     undo
U     undo all changes on this line
Ctrl-r     redo

:changes              Show changes
g;                         Jump to location of last change
g,                         Location of next change
```